Chapter3 The Relational Data Model

1 A Brief History — Why the Relational Model?

The relational model was introduced in **1970** by the English computer scientist **Edgar F. Codd** in his landmark paper *"A Relational Model of Data for Large Shared Data Banks"*. To appreciate the model's design choices, it helps to understand the problems it was solving.

Before the relational model, databases were based on **hierarchical** (tree-shaped) and **network** (graph-shaped) models. These models forced application programmers to:

- **Know the physical layout of data** — any change to file structure or storage required rewriting every application that touched that data. Codd called this *physical data dependence*.
- **Navigate the "Connection Trap"** — to retrieve related data in network models, a programmer had to walk explicit pointer paths. Missing a pointer or traversing in the wrong direction produced wrong results silently.
- **Accept inconsistency** — there was no mathematical foundation to define what "consistent" even meant, so the same fact could be stored contradictorily in different parts of the database.

Codd's 1970 paper targeted three specific goals:

- **Data independence:** Programs should work unchanged even when physical storage changes.
- **Resolving the Connection Trap:** Data relationships should be expressed declaratively (by value), not by physical pointers.
- **Mathematical consistency:** A sound basis — set theory and predicate logic — to define, query, and reason about data.

> **Key Takeaways**
>
> ⭐ The key insight was to represent all data as **tables of values**
>
> ⭐ where relationships between tables are expressed by **matching values** (foreign keys) rather than by pointers. A query language can then declare *what* data is needed, not *how* to navigate to it — this is the foundation that SQL is built on.

1.1 The Connection Trap

To understand the **Connection Trap** in legacy network databases (like the CODASYL model), imagine a database for a company with **Employees**, **Projects**, and **Departments**:

- **Department** pointers link to **Employees** assigned to that department.
- **Employee** pointers link to **Projects** that the employee works on.
- **Project** pointers link back to the **Department** funding the project.

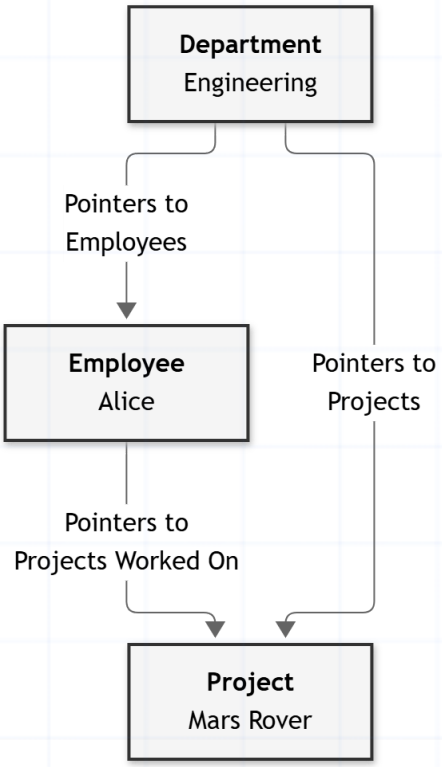


Diagram 45: Network Database Pointer Relationships — Illustrating the explicit pointer paths between Department, Employee, and Project entities that programmers had to manually navigate, creating the risk of the "Connection Trap."

🎬 SCENARIO 1: Missing a Pointer Loop / Navigation Error

suppose a manager asks a programmer:

> *"List all projects funded by the Engineering department along with the employees working on them."*

💡 Show Resolution

Where the "Connection Trap" Silent Errors Happen

Because there was no high-level query language like `SELECT ... JOIN`, the programmer had to write procedural code (in languages like COBOL) to manually move a database cursor through explicit pointer addresses step-by-step:

- The programmer starts at `Department ("Engineering")`.
- They follow the owner-to-member pointer path down to `Employee ("Alice")`.
- They follow Alice's project pointer to find `Project ("Mars Rover")`.

- **The Trap:** The programmer assumes that because they traversed `Engineering → Alice → Mars Rover`, the *Mars Rover* project must be funded by *Engineering*.

However, **Alice might work on a project funded by Marketing**.

By navigating through the *Employee-Project* pointer chain instead of checking the explicit *Department-Project* funding pointer chain, the programmer silently associates the *Mars Rover* project with *Engineering* budget reports.

Why Codd's Relational Model Solved This

Edgar F. Codd recognized that requiring programmers to act as human navigational paths meant business logic was tied directly to physical memory structures.

In a relational database, you eliminate pointer-walking entirely by joining on matching data values instead of memory pointers:

```
SELECT Department.Name, Employee.Name, Project.Name
FROM Department
JOIN Project ON Department.Dept_ID = Project.Funding_Dept_ID
JOIN Employee_Project ON Project.Project_ID = Employee_Project.Project_ID
JOIN Employee ON Employee_Project.Emp_ID = Employee.Emp_ID;
```

Here, the DBMS engine handles the physical navigation underneath, ensuring that missing or misdirected pointer paths are structurally impossible.

1.2 Physical Data Dependence

To understand **Physical Data Dependence**, imagine a company in the 1960s using a legacy mainframe file system or early hierarchical database (like IBM's IMS) to store employee records.

Imagine an enterprise payroll application where each employee record is stored on a magnetic tape or fixed-block disk as a raw, contiguous array of bytes (similar to a C `struct` or COBOL record definition).

Table 11: Original File Layout: Every employee record in the data file is laid out with fixed byte offsets

Bytes	Attribute	Example Data
00–08	Emp_ID	100249581
09–28	Name	John Doe
29–48	Address	123 Main Street
49–56	Salary	00075000

Total Record Length: 57 bytes.

The COBOL application reading this data reads the file directly by byte offset:

```
READ EMPLOYEE-FILE.
MOVE RECORD(09:20) TO EMPLOYEE-NAME.
MOVE RECORD(49:8) TO EMPLOYEE-SALARY.
```

SCENARIO 2: **The Business Change Request**

The company expands nationally and needs to add a **Phone** field right after the `Address` field. **New Record Length:** 62 bytes.

Table 12: *New File Layout*

Bytes	Attribute	Change
00–08	Emp_ID	Unchanged
09–28	Name	Unchanged
29–48	Address	Unchanged
49–53	Phone	NEW FIELD ADDED
54–61	Salary	Shifted right by 5 bytes

Show Resolution

The Pain Point: What Breaks?

Because the applications had **physical data dependence**:

1. **Broken Offsets:** Every program that reads salary now reads bytes 49–56 (which now contains part of the Phone and partial salary digits), producing corrupted data or crash errors.

2. **Cascading Code Changes:** It wasn't just the payroll program that needed updating. **Every application that ever touched the file**—such as the performance review system, tax reporting software, and benefits portal—had to be modified, recompiled, and re-tested, even if those programs didn't use the Phone field at all.
3. **Storage Hardware Upgrades:** Changing from magnetic tapes to disks, or changing disk block sizes, required modifying the low-level record-seeking routines written directly into application code.

> **How Codd's Relational Model Solved It: Physical Data Independence**
>
> Edgar F. Codd introduced an abstraction layer that decoupled application logic from physical disk layout.
>
> **In a relational database:**
>
> 1. You execute `ALTER TABLE Employee ADD COLUMN Phone VARCHAR(5);` .
> 2. Applications query data by attribute name, not by physical byte address.
> 3. The DBMS handles where bytes live on disk behind the scenes. If a new column is added, or if the database administrator changes indexes or moves the file to a different storage disk, the existing `SELECT` queries continue to work **without a single line of application code needing to be changed**.

2 Where Do Tables Come From?

Before we dive into the mathematical definition of tables, domains, and keys, we must answer a fundamental question: **Where do the tables come from?**

We don't just invent tables randomly. We design them based on how we perceive the real world. This process starts with two core concepts: **Entities** and **Relationships**.

Database design is usually a three-step process:

- **Conceptual Design (The "What"):** We create an abstract model of the real world. We identify the things that matter (entities) and how they are connected (relationships). This is often done using an Entity-Relationship (ER) diagram. This model is independent of any specific database software.
- **Logical Design (The "How"):** We take our conceptual model and map it to a specific data model, such as the Relational Model. We define tables (relations), columns (attributes), keys, and constraints. This is the main subject of this chapter.
- **Physical Design (The "Where"):** We implement our logical design on a specific Database Management System (DBMS) like MySQL, PostgreSQL, or Oracle, deciding on file organizations and indexes.

3 Mathematical Backgound

3.1 Set theory

📖 DEFINITION 1: **Set theory**

is a fundamental branch of mathematics that studies **sets**, which are well-defined collections of distinct objects called *elements* or *members*.

Tip

In computer science and database theory, set theory provides the mathematical foundation for how data is structured and queried.

Key Concepts in Set Theory:

- 1. Elements and Membership: An object x is an element of a set A, written as $x \in A$. If x is not in A, it is written as $x \notin A$.
  - Example: $A = \{1, 2, 3, 4\}$. Here, $3 \in A$, but $5 \notin A$.
- 2. Core Set Operations
  - **Union ($A \cup B$):** Combines all unique elements from both sets.
    - $\{1, 2\} \cup \{2, 3\} = \{1, 2, 3\}$
  - **Intersection ($A \cap B$):** Finds only elements shared by both sets.
    - $\{1, 2\} \cap \{2, 3\} = \{2\}$
  - **Difference ($A \setminus B$ or $A - B$):** Finds elements in set A that are not in set B.
    - $\{1, 2, 3\} - \{2, 3\} = \{1\}$
  - **Cartesian Product ($A \times B$):** The set of all ordered pairs (a, b) where $a \in A$ and $b \in B$.
    - $\{1, 2\} \times \{X, Y\} = \{(1, X), (1, Y), (2, X), (2, Y)\}$
- 3. Essential Properties
  - **Uniqueness:** A set contains no duplicate elements (e.g., $\{1, 1, 2\}$ is identical to $\{1, 2\}$).
  - **Unordered:** The sequence of elements does not matter (e.g., $\{A, B\} = \{B, A\}$).

3.2 First-Order Predicate Logic (FOPL)

📖 DEFINITION 2: **First-Order Predicate Logic (FOPL)**

is a formal mathematical language used to express statements, reason logically, and determine truth values based on rules.

> **Tip**
>
> While **propositional logic** can only deal with simple declarative statements (e.g., "It is raining" = True/False), **first-order logic** breaks statements down into objects, properties, relationships, and quantifiers.

Key Components of First-Order Logic:

- **Objects (Constants):** Specific entities in the domain of discourse (e.g., `John`, `CS101`, `5`).
- **Variables:** Symbols representing unspecified objects (e.g., x, y, z).
- **Predicates:** Functions that express properties of objects or relationships between them and evaluate to True or False:
  - Property: $\text{IsStudent}(x)$
  - Relationship: $\text{EnrolledIn}(x, y)$
- **Functions:** Expressions that map objects to another object (e.g., $\text{FatherOf}(x)$).
- **Logical Connectives:** Standard operators to combine expressions:
  - AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$), IF AND ONLY IF ($\leftrightarrow$).
- **Quantifiers:**
  - **Universal Quantifier ($\forall$):** "For all" or "For every".
  - **Existential Quantifier ($\exists$):** "There exists at least one".

> EXAMPLE 1:
>
> - **Natural English:** "Every student enrolled in CS101 has passed a prerequisite."
> - **First-Order Logic:**
>
> $$\forall x \left(\text{Student}(x) \wedge \text{EnrolledIn}(x, \text{CS101}) \rightarrow \exists y \left(\text{Prerequisite}(y) \wedge \text{Passed}(x, y)\right)\right)$$

4 The Relational Model

Proposed by E. F. Codd in 1970, the relational model provides a formal framework for database systems grounded in **set theory** and **first-order predicate logic**. Data is represented declaratively as collections of relations (tables) rather than physical pointers.

4.1 Core Concepts & Structural Architecture

Before diving into formal notation, it is essential to establish the structural hierarchy of a relational database, moving from global definitions down to the stored data state.

> 📖 DEFINITION 3: **Database Schema (S)**
>
> **Database Schema (S):** Defines the **entire database structure** as a set containing all relation schemas $\{R_1, R_2, \ldots, R_m\}$ along with the set of integrity constraints (IC) that govern them (such as Primary Keys, Foreign Keys, and Domain Constraints).
>
> $$S = (\{R_1, R_2, \ldots, R_m\}, IC)$$

> 📖 DEFINITION 4: **Relation Schema (R)**
>
> **Relation Schema (R):** The static logical definition of a single table, specifying its name, attributes, and domains:
>
> $$R(A_1 : D_1, A_2 : D_2, \ldots, A_n : D_n)$$

> 📖 DEFINITION 5: **Relation Instance**
>
> **Relation Instance ($r(R)$):** The dynamic snapshot of actual data stored in a relation at a given point in time:
>
> $$r(R) = \{t_1, t_2, \ldots, t_m\} \quad \text{where each } t_i \in D_1 \times D_2 \times \cdots \times D_n$$

4.2 Mathematical Definition

A **relation** R of degree (arity) n is defined mathematically as a subset of the Cartesian product of n domains:

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n$$

Where:

- **Domain (D_i):** A set of atomic (indivisible) values of the same type.
  - $D_{\text{Salary}} = \{x \in \mathbb{R} \mid x \geq 0\}$

- $D_{\text{Gender}} = \{\text{'M'}, \text{'F'}\}$

- **Degree (n):** The number of attributes (columns) defining the relation schema.
- **Cardinality ($|R|$):** The total number of tuples (rows) present in the relation instance.

4.3 Formal Tuple Notation

Each element of a relation R is an n-tuple t:

$$t = (v_1, v_2, \ldots, v_n) \quad \text{where } v_i \in D_i$$

Referencing Tuple Components

- **Index-Based Notation:**

$$t[i] = v_i$$

- **Attribute-Name Notation:**

$$t[A_i] \quad \text{or} \quad t.A_i$$

- **Sub-tuple (Projection) Notation:**

  For a subset of attributes $X \subseteq \{A_1, A_2, \ldots, A_n\}$:

$$t[X]$$

EXAMPLE 2: **Concrete Example Table: EMPLOYEE (الموظفون)**

Table 13: *Relation EMPLOYEE (الموظفون)*

Emp_ID(PK)	Fname	Lname	Salary	Dept_Code(FK)
101	Ahmed	Al-Mansour	75000.00	ENG
102	Bilal	Hassan	62000.00	MKT
103	Fatima	Al-Zahra	80000.00	ENG

Formal Schema Notation

$$\text{EMPLOYEE}(\text{Emp_ID}, \text{Fname}, \text{Lname}, \text{Salary}, \text{Dept_Code})$$

Cartesian Product Set Mapping

$$\text{EMPLOYEE} \subseteq D_{\text{Emp_ID}} \times D_{\text{Fname}} \times D_{\text{Lname}} \times D_{\text{Salary}} \times D_{\text{Dept_Code}}$$

For tuple $t_1 = (101, \text{'Ahmed'}, \text{'Al-Mansour'}, 75000.00, \text{'ENG'})$:

$$t_1[\text{Emp_ID}] = 101$$

$$t_1[\text{Fname}] = \text{'Ahmed'}$$

$$t_1[\{\text{Fname}, \text{Lname}\}] = (\text{'Ahmed'}, \text{'Al-Mansour'})$$

Table 14: *Mathematical Mapping Summary*

Relational Term	Formal Mathematical Term	Database Term	Example Value
Relation	Set R	Table	`EMPLOYEE`
Attribute	Variable / Set Dimension A_i	Column / Field	`Fname`, `Salary`
Tuple	Element $t \in R$	Row / Record	$t_1 = (101, \text{'Ahmed'}, \text{'Al-Mansour'}, 75000.00, \text{'ENG'})$
Domain	Set D_i	Data Type / Domain	$D_{\text{Salary}} = \{x \in \mathbb{R} \mid x \geq 0\}$
Degree	Arity (n)	Column Count	**5**
Cardinality	Set Size ($\lvert R \rvert$)	Row Count	**3**

4.4 Properties of Relations

Table 15: *Properties of Relations*

Property	Formal Relation	SQL Table
No duplicate tuples	Required by set theory	Enforced only with PK or UNIQUE
Tuple ordering	Irrelevant	Undefined without ORDER BY
Attribute ordering	Irrelevant	Fixed by column position
Atomic attribute values	Required (**1NF**)	Required in normalized tables
NULL values	Not allowed (formal model)	Allowed in ANSI SQL

> **Note**
>
> SQL tables are **multisets** (bags), not sets. Without a primary key, SQL allows duplicate rows. The formal relational model does not. Always define a primary key.

4.5 Domain Specification Components

> **DEFINITION 6: Domain**
>
> A domain D is a set of atomic (indivisible) values. Each domain has a logical name, data type, and format.

Table 16: *Domain Specification Components*

Component	What it Specifies	SQL Mechanism	Example
1. Data Type	The kind of values (integer, text, date ...)	`INT`, `VARCHAR`, `DATE`, `DECIMAL`, `BOOLEAN`	`Salary DECIMAL(10,2)`
2. Length / Precision	Maximum characters, total digits, decimal places	`VARCHAR(n)`, `CHAR(n)`, `DECIMAL(p,s)`	`Name VARCHAR(50)`, `GPA DECIMAL(3,2)`
3. Date Format	How date/time values are interpreted	`DATE` (ISO 8601: `YYYY-MM-DD` internally)	`Bdate DATE` stores `1995-08-20`
4. Range	Lower and upper bounds on numeric or date values	`CHECK(col BETWEEN a AND b)`	`CHECK(GPA BETWEEN 0.0 AND 4.0)`
5. Constraints	Rules that restrict allowed values beyond type	`NOT NULL`, `UNIQUE`, `CHECK(expr)`, `PRIMARY KEY`, `FOREIGN KEY`	`CHECK(Gender IN ('M','F'))`
6. NULL Support	Whether the attribute may be absent	`NULL` (default) or `NOT NULL`	`Middle_Name VARCHAR(30)` — nullable
7. Default Value	Value used when none is supplied on INSERT	`DEFAULT expr`	`Status VARCHAR(10) DEFAULT 'Active'`

4.5.1 Data Type

Databases provide support for different types of data and their variants:

- INTEGER:- Whole numbers (e.g., 1, 42, -17)
- FLOAT/ DECIMAL - Real numbers with decimal points.
- VARCHAR/ TEXT - Character strings.
- DATE- Calendar dates.
- BOOLEAN- True/False values.
- ENUM- Enumerated list of values

4.5.2 Length / Precision

Length means the number of characters or digits that an attribute value has.

- For VARCHAR(50): Maximum 50 characters.
- For DECIMAL(10,2): Total 10 digits, 2 after decimal point.
- For CHAR(6): Exactly 6 characters (e.g., PIN code).

4.5.3 Date Format

A date contains day, month, and year in combination:

- DD/MM/YYYY - 07/02/2026 (European format)
- MM/DD/YYYY - 02/07/2026 (American format)
- YYYY-MM-DD - 2026-02-07 (ISO 8601 standard)

4.5.4 Range

A range is specified by lower and upper bounds of data values:

- GPA: [0.0, 4.0]
- Age: [0, 150]
- Percentage: [0, 100]
- Temperature: [-273.15, ∞) (Celsius, above absolute zero)

4.5.5 Constraints

Particular conditions that put restrictions on allowed values:

- NOT NULL - Value must be provided
- UNIQUE - No duplicates allowed
- CHECK - Custom validation rule
- PRIMARY KEY - Uniquely identifies each row
- FOREIGN KEY - References another table

4.5.6 NULL Support

NULL represents *unknown*, *missing*, or *not applicable*. Different NULLs express different real-world meanings:

Table 17: *NULL Values meanings*

Reason for NULL	Example
Unknown	`Bdate` of an employee not on record
Not applicable	`Super_ssn` for the CEO (no supervisor)
Not yet known	`Mgr_start_date` during a transition period

4.5.7 Default Value

If nothing is entered, database assigns a default value:

- Status: DEFAULT 'Active'
- Created_Date: DEFAULT CURRENT_DATE
- Country: DEFAULT 'USA'
- Is_Active: DEFAULT TRUE

4.6 Keys and Integrity Constraints

While the mathematical definition of a relation (Section 5.2) tells us how data is structured, it does not tell us how to ensure the data is **accurate, unique, and meaningfully connected**. To enforce business rules and prevent data anomalies, the relational model relies on **Keys** (to identify records) and **Integrity Constraints** (to enforce rules).

4.6.1 Keys: Identifying Tuples

In the real world, we identify entities by unique characteristics (e.g., a person by their National ID, a book by its ISBN). In the relational model, we use **Keys** to uniquely identify tuples (rows) within a relation.

Note

Keys are used to uniquely identify tuples (rows) within a relation.

4.6.1.1 Super Key

DEFINITION 7: Super Key

A set of one or more attributes (columns) that uniquely identifies a row (tuple) within a table. Any combination of columns that prevents duplicate rows is a super key.

Mathematical Definition

Let RR be a relation schema. A set of attributes SK⊆R is a super key if, for any valid state r(R), no two distinct tuples t_1 and t_2 can have the same values for SKSK.

- *Equation:*

$$\forall t_1, t_2 \in r(R), \quad (t_1 \neq t_2) \implies t_1[SK] \neq t_2[SK]$$

where:

- $r(R)$: A valid state (instance) of the relation schema R.
- t_1, t_2: Two distinct tuples (rows) within the relation state $r(R)$.
- SK: The set of attributes that form the super key ($SK \subseteq R$).
- $t[SK]$: The projection (values) of tuple t on the set of attributes SK

EXAMPLE 3: Understanding the Super Key Rule

Table 18: *Relation State: STUDENT (r(R)*

Student_ID	National_ID	Email	Name	Department
001	1029384756	ahmed.k@univ.edu.ps	Ahmed Khalil	Computer Science
002	1029384757	tariq.s@univ.edu.ps	Tariq Mansour	Computer Science
003	1029384758	sara.a@univ.edu.ps	Sara Al-Ahmad	Information Technology
004	1029384759	ahmed.k2@univ.edu.ps	Ahmed Khalil	Software Engineering

A **Super Key (SK)** is any combination of attributes (columns) that uniquely identifies a row in a table. This means if you know the value of SK, no two students in the entire table can share that exact value.

Examples from the Table Above

1. Single-Attribute Super Keys

These columns are naturally unique by themselves:

- $\{\text{Student_ID}\}$: Every student has a unique university ID number.
- $\{\text{National_ID}\}$: Every student has a unique national ID number (رقم الهوية).
- $\{\text{Email}\}$: Every university email address is unique.

2. Composite Super Keys (Supersets)

A Super Key remains a Super Key even if you add extra, unnecessary attributes to a unique key:

- $\{\text{Student_ID}, \text{Name}\}$: Unique because `Student_ID` is already unique.
- $\{\text{National_ID}, \text{Department}\}$: Unique because `National_ID` is already unique.
- $\{\text{Email}, \text{Name}, \text{Department}\}$: Unique because `Email` is already unique.
- $\{\text{Student_ID}, \text{National_ID}, \text{Email}, \text{Name}, \text{Department}\}$: The entire relation schema is also a valid Super Key.

3. NOT Super Keys

These attributes cannot uniquely identify a row:

- $\{\text{Name}\}$: Fails because two different students share the same name ("Ahmed Khalil").
- $\{\text{Department}\}$: Fails because multiple students belong to "Computer Science".
- $\{\text{Name}, \text{Department}\}$: Fails if two students named "Ahmed Khalil" ever enroll in the same department.

4.6.1.2 Candidate Key

📖 DEFINITION 8: **Candidate Key**

A **Candidate Key** is a *minimal* Super Key. It has the exact same unique identification property as a Super Key, but it contains **no redundant attributes**. If you remove any attribute from a Candidate Key, it ceases to be a key.

Mathematical Definition

A set of attributes $CK \subseteq R$ is a **Candidate Key** for a relation schema R if and only if it satisfies the following two conditions:

1. CK is a super key.
2. Removing any attribute A from CK results in a set $X = CK \setminus \{A\}$ that is no longer capable of uniquely identifying every tuple (it ceases to be a Super Key).

EXAMPLE 4:

Table 19: *Relation State: STUDENT (r(R))*

Student_ID	National_ID	Email	Name	Department
001	1029384756	ahmed.k@univ.edu.ps	Ahmed Khalil	Computer Science
002	1029384757	tariq.s@univ.edu.ps	Tariq Mansour	Computer Science
003	1029384758	sara.a@univ.edu.ps	Sara Al-Ahmad	Information Technology
004	1029384759	ahmed.k2@univ.edu.ps	Ahmed Khalil	Software Engineering

A **Candidate Key (CK)** is a **minimal Super Key**. To be a Candidate Key, a set of attributes must satisfy two rules:

1. **Uniqueness:** It can uniquely identify every row (tuple) in the table.
2. **Minimality (Irreducibility):** If you remove *any* attribute from it, it loses its uniqueness property.

$$\forall A \in CK, \quad CK \setminus \{A\} \quad \text{is no longer a Super Key}$$

Analysis of the Table Attributes

1. Valid Candidate Keys

These single attributes are Candidate Keys because each one uniquely identifies every student, and they cannot be reduced further:

- $CK_1 = \{\text{Student_ID}\}$: Every student has a unique university ID.
- $CK_2 = \{\text{National_ID}\}$: Every student has a unique national ID (رقم الهوية).
- $CK_3 = \{\text{Email}\}$: Every university email address is assigned to exactly one student.

Each of these is a valid candidate to be chosen as the table's **Primary Key**.

2. Super Keys that are NOT Candidate Keys

Consider these combinations:

- $SK_1 = \{\text{Student_ID}, \text{Name}\}$
  - *Is it unique?* Yes, because `Student_ID` is unique.
  - *Is it minimal?* **No.** If you remove `Name`, you are left with $\{\text{Student_ID}\}$, which is still unique. Therefore, `Name` is redundant.
- $SK_2 = \{\text{National_ID}, \text{Email}\}$
  - *Is it unique?* Yes.
  - *Is it minimal?* **No.** Removing `National_ID` leaves $\{\text{Email}\}$, which is already unique on its own.

3. Attributes that are NOT Candidate Keys

- $\{\text{Name}\}$: Fails uniqueness (two students are named "Ahmed Khalil").
- $\{\text{Department}\}$: Fails uniqueness (multiple students belong to "Computer Science").
- $\{\text{Name}, \text{Department}\}$: Fails uniqueness because two students with the same name could potentially enroll in the same department.

4.6.1.3 Primary Key

📖 **DEFINITION 9: Primary Key**

The specific candidate key chosen by the database designer to uniquely identify each row in the table. It must contain unique values and cannot contain `NULL` values.

ⓘ **Mathematical Definition**

A set of attributes $PK \subseteq R$ is a **Primary Key** for a relation schema R if and only if for every valid relation state $r(R)$, it satisfies:

1. Uniqueness (Super Key Property)

$$\forall t_1, t_2 \in r(R), \quad (t_1 \neq t_2) \implies t_1[PK] \neq t_2[PK]$$

2. Entity Integrity Constraint (Non-Null Property)

$$\forall t \in r(R), \quad \forall A \in PK, \quad t[A] \neq \text{NULL}$$

(If PK consists of a single attribute $PK = \{A\}$, this simplifies to $\forall t \in r(R),\ t[PK] \neq \text{NULL}$.)

EXAMPLE 5:

Table 20: *Relation State: FACULTY_DIRECTORY (r(R))*

Staff_ID	National_ID	Name	Email	Department
101	901234567	Dr. Tariq Al-Hassan	tariq.h@univ.edu.ps	Computer Systems Eng.
102	901234568	Eng. Reem Mansour	reem.m@univ.edu.ps	Information Technology
103	901234569	Dr. Omar Khalid	NULL	Computer Systems Eng.
104	901234570	Layla Suleiman	NULL	Computer Science

Why $\underline{\text{Staff_ID}}$ is a Valid Primary Key:

1. **Uniqueness (Super Key Property):** Every employee has a unique `Staff_ID` (`101`, `102`, `103`, `104`). No two distinct rows share the same ID.
2. **Entity Integrity (Non-Null Property):** Every employee is guaranteed to have a `Staff_ID`. No row has a `NULL` value for this attribute.

Because both conditions are strictly satisfied across the relation, $\underline{\text{Staff_ID}}$ safely serves as the **Primary Key**.

Illustrating the Email Case (Why NULL Invalidates a Primary Key)

Even if all non-null email addresses are unique, `Email` fails as a Primary Key due to the following mathematical and logical violations:

1. Violation of the Entity Integrity Constraint

By definition, a Primary Key PK must satisfy:

$$\forall t \in r(R), \quad t[PK] \neq \text{NULL}$$

In this table, for tuple t_3 (Dr. Omar Khalid) and tuple t_4 (Layla Suleiman):

$$t_3[\text{Email}] = \text{NULL}$$

$$t_4[\text{Email}] = \text{NULL}$$

Because $t_3[\text{Email}] = \text{NULL}$, the **Entity Integrity Constraint is broken**.

2. Loss of Row Identification

In relational theory, `NULL` represents an **unknown** or **missing** value. Because `Email` is missing for both t_3 and t_4, the database cannot use `Email` to distinguish Dr. Omar Khalid from Layla Suleiman.

3. Failure of Mathematical Equality

In standard relational calculus and SQL three-valued logic, `NULL = NULL` evaluates to `UNKNOWN` (not `TRUE`). Therefore, testing uniqueness on `NULL` attributes fails the formal condition:

$$\forall t_1, t_2 \in r(R), \quad (t_1 \neq t_2) \implies t_1[PK] \neq t_2[PK]$$

Practical Takeaway

- `Staff_ID` remains the valid **Primary Key** ($\underline{\text{Staff_ID}}$) because it is both unique and strictly non-null ($\neq \text{NULL}$) for every tuple.
- `Email` can only be classified as an optional attribute or a candidate attribute with a sparse/filtered unique index, but **it can never be a Primary Key** whenever `NULL` entries exist.

Extending the Concept: Alternative (Alternate) Keys

In our `FACULTY_DIRECTORY` table, `National_ID` is also a completely valid Candidate Key because it is both **unique** for every single faculty member and **guaranteed to never be NULL** ($\forall t \in r(R),\ t[\text{National_ID}] \neq \text{NULL}$).

However, standard relational theory dictates that a relation schema R can have **exactly one Primary Key** (PK).

Definition: Alternate (or Alternative) Key

Any **Candidate Key** that is **not** chosen by the database designer to be the Primary Key is called an **Alternate Key** (or **Secondary Unique Key**).

Mathematical Relation

$$\text{Alternate Keys} = \text{Candidate Keys} \setminus \{\text{Primary Key}\}$$

Applied Example

From our pool of valid Candidate Keys for the faculty directory:

$$\text{Candidate Keys} = \{\{\text{Staff_ID}\}, \{\text{National_ID}\}\}$$

1. **Primary Key (PK):** We select $\underline{\text{Staff_ID}}$ to serve as the main entity identifier.
2. **Alternate Key (AK):** National_ID becomes the Alternate Key.

4.6.1.4 Foreign Key

📖 DEFINITION 10: **Foreign Key**

An attribute or set of attributes in one table that references the primary key (or candidate key) of another table, establishing a logical link and maintaining referential integrity between them.

Mathematical Definition

Let R_1 and R_2 be two relation schemas (where R_1 and R_2 can be the same relation in a self-reference), and let PK be the **Primary Key** of R_2.

A set of attributes $FK \subseteq R_1$ is a **Foreign Key** in R_1 referencing R_2 if it satisfies two conditions for every valid relational database state $db = \{r(R_1), r(R_2)\}$:

1. Domain Compatibility

$$\text{Dom}(FK) = \text{Dom}(PK)$$

(The attributes in FK have the same domain/data type as the primary key PK of R_2. If FK and PK are composite, there must be a one-to-one mapping between their corresponding attributes).

2. Referential Integrity Constraint

$$\forall t_1 \in r(R_1), \quad (\exists t_2 \in r(R_2) \quad \text{such that} \quad t_1[FK] = t_2[PK]) \quad \vee \quad (t_1[FK] = \text{NULL})$$

Key Properties:

- **Referencing vs. Referenced:** R_1 is called the **referencing relation**, and R_2 is the **referenced relation**.
- **Nullability:** Unlike Primary Keys, Foreign Key attributes **can contain NULL values** (unless explicitly constrained by a `NOT NULL` constraint), representing an unassigned or missing relationship.
- **Self-Referencing Foreign Key:** When $R_1 = R_2$, the foreign key references its own table's primary key (e.g., `Super_ssn` referencing `Ssn` within an `EMPLOYEE` relation).

EXAMPLE 6: **University Course Enrollment System to illustrate how a Foreign Key (FK) works**

To store information about university departments and their students without repeating data, we split the information into two relations:

1. `DEPARTMENT` (The **Referenced Relation** R_2)
2. `STUDENT` (The **Referencing Relation** R_1)

Table 21: *The Referencing Relation: STUDENT*

Table 22: *The Referenced Relation: DEPARTMENT*

Student_ID (PK)	Name	Major	Dept_II (FK)	Dept_ID (PK)	Dept_Name	Building
2024001	Ahmed Khalil	Hardware Engineering	10	10	Computer Systems Engineering	Engineering Building
2024002	Tariq Mansour	Network Security	20	20	Information Technology	IT Complex
2024003	Sara Al-Ahmad	Data Science	30	30	Computer Science	Main Campus
2024004	Reem Suleiman	Undecided	NULL			

For a Foreign Key $FK \subseteq \text{STUDENT}$ referencing DEPARTMENT:

$$\pi_{\text{Dept_ID}}(\text{STUDENT}) \setminus \{\text{NULL}\} \subseteq \pi_{\text{Dept_ID}}(\text{DEPARTMENT})$$

Let's check the values in `STUDENT.Dept_ID`:

$$\{10, 20, 30, \text{NULL}\} \setminus \{\text{NULL}\} = \{10, 20, 30\}$$

Since $\{10, 20, 30\} \subseteq \{10, 20, 30\}$ (the Primary Keys present in `DEPARTMENT`), the referential integrity constraint holds true.

Key Observations from This Example

1. Valid Linkage

Student `2024001` (Ahmed Khalil) has `Dept_ID = 10`. The database looks at `DEPARTMENT` with `Dept_ID = 10` and knows Ahmed belongs to **Computer Systems Engineering**.

2. Nullability (Allowed)

Student `2024004` (Reem Suleiman) has `Dept_ID = NULL`.

- **Why it's valid:** Unlike Primary Keys, Foreign Keys **can be NULL** if a relationship is optional (e.g., a newly enrolled student who hasn't declared a department yet).

3. Referential Integrity Violation (Prevented by the DBMS)

If you try to insert a new student with an invalid department:

```
INSERT INTO STUDENT VALUES ('2024005', 'Omar Khalid', 'AI', 99);
```

- **Result: REJECTED BY DBMS.**
- **Reason:** `Dept_ID = 99` does not exist in `DEPARTMENT`. The foreign key constraint prevents "orphan" references to non-existent departments.

4.6.2 Integrity Constraints (ICs)

> 📖 DEFINITION 11: **Integrity Constraints**
>
> Integrity Constraints are business rules applied to the relational schema to ensure the accuracy and consistency of the data. If a database state violates an integrity constraint, it is considered an **invalid state**.

4.6.2.1 Domain Constraints

These are the most basic constraints. They ensure that every attribute value must belong to its defined domain (data type, format, and range).

- **Rule:** Values must match the specified data type and fall within allowed ranges.
- **Example:** If the domain of `Age` is defined as integers between 18 and 100, inserting a value of `150` or `'Twenty'` violates the domain constraint.

4.6.2.2 Key Constraints & Entity Integrity

This constraint governs the Primary Key. Because the Primary Key's entire purpose is to uniquely identify a row, it must always have a value.

- **Rule (Entity Integrity):** No part of a Primary Key can be `NULL`.
- **Example:** You cannot insert an employee into the `EMPLOYEE` table without an `Emp_ID`. If `Emp_ID` is `NULL`, the database cannot uniquely identify or reference that specific employee.
- *Note:* Uniqueness is also enforced (no two rows can have the same PK), which is inherently guaranteed by the definition of a Primary Key.

4.6.2.3 Referential Integrity

This constraint governs Foreign Keys. It ensures that relationships between tables remain consistent. You cannot have an "orphan" record pointing to a non-existent parent.

- **Rule:** A Foreign Key value must either be completely `NULL` (meaning the relationship doesn't exist yet) OR it must exactly match an existing Primary Key value in the referenced table.
- **Example:** If the `DEPARTMENT` table only has departments `ENG`, `MKT`, and `HR`, you **cannot** insert an employee with `Dept_Code = 'FIN'`. The database will reject the insertion to maintain referential

integrity.

4.7 Formal Schema Notation

In database literature and practice, there are two primary ways to express a relational schema:

1. **The Strict Formal Notation** (The Elmasri & Navathe academic standard).
2. **The Shorthand Relational Notation** (The practical, inline design standard).

4.7.1 Style 1 — The Strict Formal Notation (Elmasri & Navathe Style)

This notation is used in pure database theory. It strictly separates the **structural definition** of the tables from the **Integrity Constraints (ICs)**. It focuses on the mathematical *state* of the database, not the procedural *actions* of the DBMS.

The 5 Rules of Strict Formal Notation:

- **Domains are Mandatory:** Every attribute must be followed by a colon and its domain (data type).
  - *Format:* `R(A1:D1, A2:D2, ...)`
- **Primary Keys are Underlined:** The primary key attribute(s) are denoted by a solid underline. If it is a composite key, *every* attribute in the key is underlined.
  - *Format:* `R(` A1 `, A2, ...)`
- **Foreign Keys are NOT Marked Inline:** Foreign keys are written as normal attributes inside the parentheses. Their status as foreign keys is declared **only** in the Integrity Constraint (IC) list below the schema.
- **Integrity Constraints (ICs) are Listed Separately:** All rules regarding Foreign Keys (Referential Integrity), NOT NULL, and CHECK constraints are written sequentially below the relation as `IC1, IC2, IC3 ...`
- **Referential Integrity uses Subset Notation:** Foreign keys are expressed using the subset symbol (⊆).
  - *Format:* `R1.FK_Attribute` ⊆ `R2.PK_Attribute`

> **EXAMPLE 7: The Core Company Hierarchy (Simple Keys & Basic Constraints)**
>
> This schema models the foundational relationship between a company's departments and its employees. Every department must have a unique number and a non-null name, and is managed by a specific employee. Every employee has a unique SSN, works in one department, and reports to a supervisor. We must enforce a minimum salary threshold and ensure referential integrity between employees and their departments/supervisors.

$$\text{DEPARTMENT}(\underline{\text{Dnumber}} : \text{Integer}, \text{Dname} : \text{String},$$
$$\text{Mgr_ssn} : \text{String}, \text{Mgr_start_date} : \text{Date})$$
$$\text{IC1: Dname} \neq \text{NULL}$$
$$\text{IC2: Mgr_ssn} \subseteq \text{EMPLOYEE.Ssn}$$

$$\text{EMPLOYEE}(\underline{\text{Ssn}} : \text{String}, \text{Fname} : \text{String},$$
$$\text{Lname} : \text{String}, \text{Salary} : \text{DECIMAL}(10,2),$$
$$\text{Dno} : \text{Integer}, \text{Super_ssn} : \text{String})$$
$$\text{IC3: Fname} \neq \text{NULL}$$
$$\text{IC4: Lname} \neq \text{NULL}$$
$$\text{IC5: } Salary \geq 30000$$
$$\text{IC6: Dno} \subseteq \text{DEPARTMENT.Dnumber}$$
$$\text{IC7: Super_ssn} \subseteq \text{EMPLOYEE.Ssn}$$

$$S = \{\text{DEPARTMENT}, \text{EMPLOYEE}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}, \text{IC4}, \text{IC5}, \text{IC6}, \text{IC7}\}$$

📝 EXAMPLE 8: **Project Management and Assignments (Composite Keys)**

This schema models how employees are assigned to various projects. A project is uniquely identified by its number and is controlled by a specific department. The actual assignment of an employee to a project (and the hours they work) is tracked in a separate relation. The `WORKS_ON` relation uses a **Composite Primary Key** made entirely of two Foreign Keys. We also enforce a valid mathematical range for the hours worked.

$$\text{PROJECT}(\underline{\text{Pnumber}} : \text{Integer}, \text{Pname} : \text{String},$$
$$\text{Plocation} : \text{String}, \text{Dnum} : \text{Integer})$$
$$\text{IC1: Pname} \neq \text{NULL}$$
$$\text{IC2: Dnum} \subseteq \text{DEPARTMENT.Dnumber}$$

$$\text{WORKS_ON}(\underline{\text{Essn}} : \text{String}, \underline{\text{Pno}} : \text{Integer},$$
$$\text{Hours} : \text{DECIMAL}(10,2))$$
$$\text{IC3: } 0.5 \leq Hours \leq 50.0$$
$$\text{IC4: Essn} \subseteq \text{EMPLOYEE.Ssn}$$
$$\text{IC5: Pno} \subseteq \text{PROJECT.Pnumber}$$

$$S = \{\text{PROJECT}, \text{WORKS_ON}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}, \text{IC4}, \text{IC5}\}$$

📝 EXAMPLE 9: **Employee Dependents (Set Constraints & Weak Entities)**

This schema models the dependents (spouse, children) of employees for insurance purposes. Because a dependent's name alone isn't unique across the whole company, they are identified by the combination of the Employee's SSN and the Dependent's name. This creates a **Composite Primary Key**. We also restrict the "Relationship" attribute to a specific set of allowed string values.

$$\begin{aligned}
&\text{DEPENDENT}(\underline{\text{Essn}} : \text{String},\\
&\quad \underline{\text{Dependent_name}} : \text{String},\\
&\quad \text{Gender} : \text{String}, \text{Bdate} : \text{Date},\\
&\quad \text{Relationship} : \text{String})\\
&\text{IC1: } \mathit{Gender} \in \{\text{'}M\text{'}, \text{'}F\text{'}\}\\
&\text{IC2: } \mathit{Relationship} \in \{\text{'}\mathit{Spouse}\text{'}, \text{'}\mathit{Son}\text{'}, \text{'}\mathit{Daughter}\text{'}\}\\
&\text{IC3: Essn} \subseteq \text{EMPLOYEE.Ssn}\\
&S = \{\text{DEPENDENT}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}\}
\end{aligned}$$

4.7.2 The Shorthand Relational Schema Notation

This notation is used in practical database design, logical modeling, and university exams where writing out domains and massive IC lists is too time-consuming. It brings the structural relationships **inline** with the attributes.

The 4 Rules of Shorthand Notation:

- **Drop the Domains:** Data types (`:String` , `:Integer`) are omitted to save space.
- **Keep Primary Keys Underlined:** The solid underline for Primary Keys (simple and composite) remains exactly the same as the formal style.
- **Bring Foreign Keys Inline:** Foreign keys are denoted using an inline arrow (→) or inline subset (⊆) directly after the attribute name, pointing to the referenced table and primary key.
  - *Format:* `FK_Attribute` → `REFERENCED_TABLE(PK_Attribute)`
- **Composite Foreign Keys use Parentheses:** If multiple attributes form a composite foreign key, wrap them in parentheses before the arrow.
  - *Format:* `(FK1, FK2)` → `REFERENCED_TABLE(PK1, PK2)`

📝 EXAMPLE 10: **University Course Registration System (Comprehensive Shorthand)**

This schema models students enrolling in university courses. We use the shorthand notation to map the relationships directly inline to keep the design compact. A student has a unique ID and a GPA restricted between 0.0 and 4.0. Courses are offered by specific departments. Enrollments link students

and courses, creating a composite primary key. *(Note:* `NN` *denotes NOT NULL,* `U` *denotes UNIQUE, and* `CK` *denotes CHECK constraint, which are common additions to shorthand notation).*

$$\begin{aligned}&\text{DEPARTMENT}(\underline{\text{DeptCode}}, \text{DeptName NN U},\\&\quad \text{Building})\end{aligned}$$

$$\begin{aligned}&\text{STUDENT}(\underline{\text{StudentID}}, \text{Fname NN},\\&\quad \text{Lname NN}, \text{Email NN U},\\&\quad \text{GPA CK } (\text{GPA BETWEEN } 0.0 \text{ AND } 4.0),\\&\quad \text{SDepartment} \subseteq \text{DEPARTMENT.DeptCode})\end{aligned}$$

$$\begin{aligned}&\text{COURSE}(\underline{\text{CourseID}}, \text{Title NN},\\&\quad \text{Credits CK } (\text{Credits} > 0),\\&\quad \text{OfferedBy} \subseteq \text{DEPARTMENT.DeptCode})\end{aligned}$$

$$\begin{aligned}&\text{ENROLLMENT}(\underline{\text{StudentID}} \subseteq \text{STUDENT.StudentID},\\&\quad \underline{\text{CourseID}} \subseteq \text{COURSE.CourseID},\\&\quad \text{Semester}, \text{Grade})\end{aligned}$$

5 The ER-to-Relational Mapping Algorithm

The Entity-Relationship (ER) model is a **conceptual** tool used to design a database from a business perspective. However, a DBMS cannot execute an ER diagram. To implement the database, we must translate (map) the conceptual ER model into a **logical** Relational Schema (tables).

This translation follows a strict, step-by-step algorithm. If followed correctly, the resulting relational schema will inherently satisfy **Third Normal Form (3NF)**, preventing data redundancy and update anomalies.

5.1 Regular (Strong) Entities and Simple Attributes

The Rule: Map each strong entity type to a relation (table). Include all simple attributes. Choose one of the candidate keys as the Primary Key (PK).

- **Composite Attributes:** Flatten them. Include only the primitive sub-attributes as separate columns.
- **Derived Attributes: Discard them.** They are computed on the fly in SQL and never stored as columns.

📝 EXAMPLE 11:

`EMPLOYEE` entity with simple attributes `Emp_ID` (Key), `Fname`, `Lname`. Composite attribute `Address` (Street, City, Zip). Derived attribute `Age` (from Bdate).

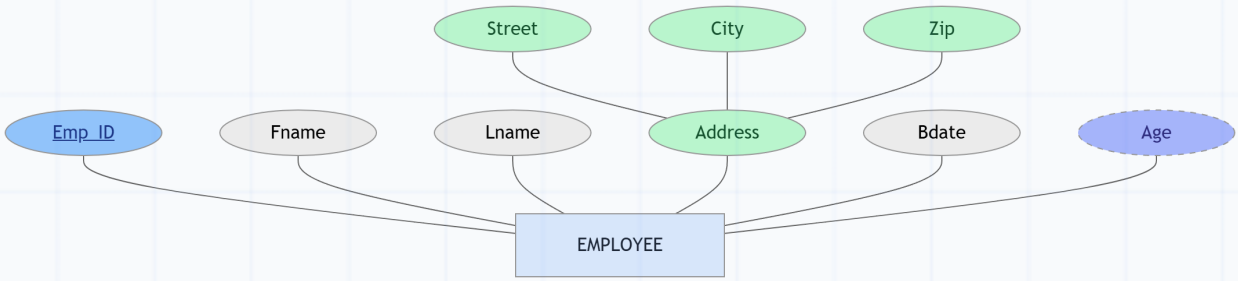


Diagram 46: EMPLOYEE entity with simple attributes Emp_ID (Key), Fname, Lname. Composite attribute Address (Street, City, Zip). Derived attribute Age (from Bdate).

✅ Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}}, \text{Fname}, \text{Lname}, \text{Street}, \text{City}, \text{Zip}, \text{Bdate})$$

> **Note**
>
> `Address` *is flattened into Street, City, Zip.* `Age` *is dropped.*

5.2 Weak Entities

The Rule: Map the weak entity to a new relation. The Primary Key of this new relation is the **Composite Key** consisting of the owner entity's PK + the weak entity's partial key (discriminator). Include the owner's PK as a Foreign Key (FK).

📝 EXAMPLE 12:

`DEPENDENT` is a weak entity of `EMPLOYEE`. Partial key is `Dependent_Name`.

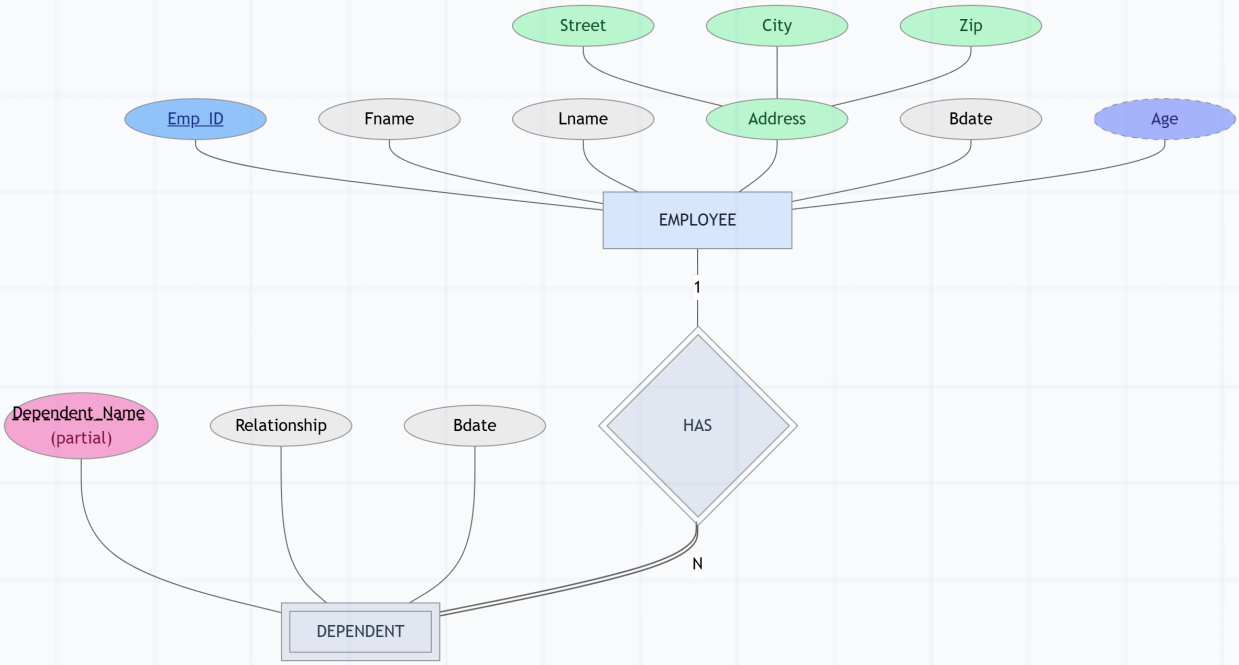


Diagram 47: DEPENDENT is a weak entity of EMPLOYEE. Partial key is Dependent_Name.

✅ Show Solution

$$\begin{aligned}&\text{DEPENDENT}(\underline{\text{Emp_ID}} : \text{Integer},\\ &\qquad \underline{\text{Dependent_Name}} : \text{String},\\ &\qquad \text{Relationship} : \text{String}, \text{Bdate} : \text{Date})\\ &\text{IC1: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}\end{aligned}$$

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer})$$

$$S = \{\text{DEPENDENT}, \text{EMPLOYEE}\} \cup \{\text{IC1}\}$$

> **Note**
>
> `Emp_ID` *is underlined because it is part of the composite PK, and it also acts as the FK to EMPLOYEE.*

5.3 Multivalued Attributes

The Rule: Multivalued attributes cannot be mapped as a single column (violating 1NF). Create a **new relation** to represent it. The PK of this new relation is the combination of the owning entity's PK and the multivalued attribute itself.

EXAMPLE 13:

`EMPLOYEE` has a multivalued attribute `Degree_Certifications`.

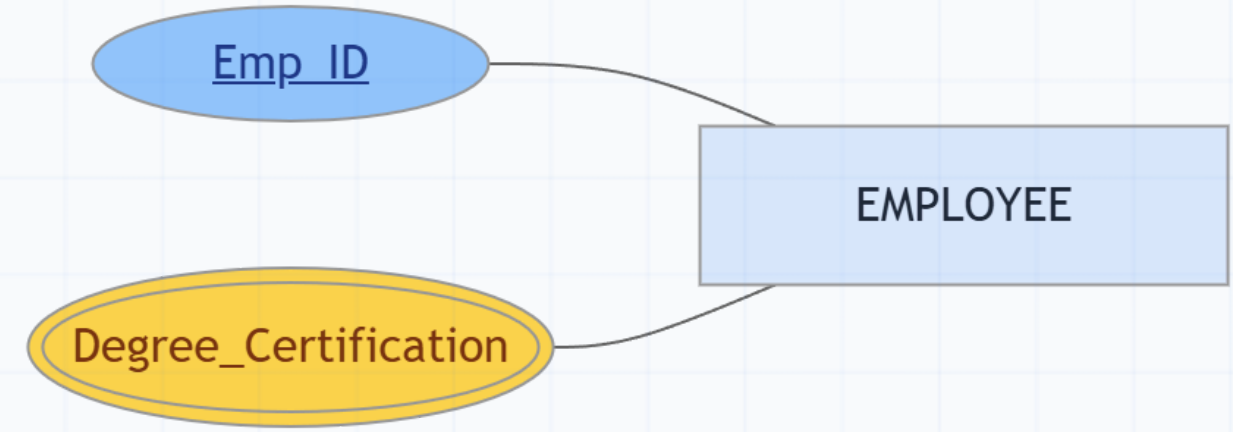


Diagram 48: EMPLOYEE has a multivalued attribute Degree_Certifications.

✅ Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer})$$

$$\text{EMPLOYEE_DEGREES}(\underline{\text{Emp_ID}} : \text{Integer}, \underline{\text{Degree_Certification}})$$

$$\text{IC1: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$$

$$S = \{\text{EMPLOYEE}, \text{EMPLOYEE_DEGREES}\} \cup \{\text{IC1}\}$$

> ⓘ **Note**
>
> *This creates a 1:N relationship implicitly between EMPLOYEE and EMPLOYEE_DEGREES.*

5.4 Mapping Relationships, Relationship Attributes, and Participation

When translating an Entity-Relationship (ER) diagram into a Relational Schema, the **cardinality** (1:N, 1:1, M:N) dictates the *structural mapping* (where tables and foreign keys go). However, the **participation constraint** (Total vs. Partial) dictates the *integrity constraints* (specifically, whether a Foreign Key can be `NULL`).

- **Total Participation (Double line in ER):** Every instance of the entity **must exist** in the relationship. (e.g., "Every order *must* have a customer").
- **Partial Participation (Single line in ER):** An instance of the entity **may or may not exist** in the relationship. (e.g., "A customer *may or may not* have placed an order").

Below are the comprehensive rules and cases for mapping all binary relationship scenarios, including how to handle relationship attributes.

5.4.1 Case 1: 1:N Relationship (Partial Participation on N-side) + Rel Attribute

Rules to Apply:

- **Structural Rule:** Add the 1-side PK as a Foreign Key in the N-side table.
- **Attribute Rule:** Place the relationship attribute in the N-side table.
- **Constraint Rule:** Because the N-side is Partial, the FK **may or may not** have a value (it is nullable).

The Structural Rule: Use the **Foreign Key approach**. Take the Primary Key of the "1-side" (parent) entity and add it as a Foreign Key attribute in the "N-side" (child) entity's relation. Relationship attributes are also placed on the N-side.

The Participation Impact:

- **Partial Participation on the N-side:** The Foreign Key in the N-side table is allowed to be `NULL`.
  - *Example:* An `EMPLOYEE` may or may not be assigned to a `DEPARTMENT`. The `Dept_ID` FK in `EMPLOYEE` is nullable.
- **Total Participation on the N-side:** The Foreign Key in the N-side table **cannot** be `NULL`. You must add a `NOT NULL` constraint to the FK.
  - *Example:* Every `ORDER` *must* belong to a `CUSTOMER`. The `Cust_ID` FK in `ORDER` is `NOT NULL`.
- **Total Participation on the 1-side:** This does not change the FK placement, but it implies a business rule: every record in the 1-side table *must* be referenced by at least one record in the N-side table. (This is typically enforced by application logic, as standard SQL lacks a direct "must be referenced by at least one child" constraint).

> 📝 **EXAMPLE 14:**
>
> A `DEPARTMENT` (1) manages `EMPLOYEE`s (N). The relationship `WORKS_FOR` has a relationship attribute `Start_Date`. An employee **may or may not** be assigned to a department yet (e.g., a new hire in orientation). A department **may or may not** have employees.

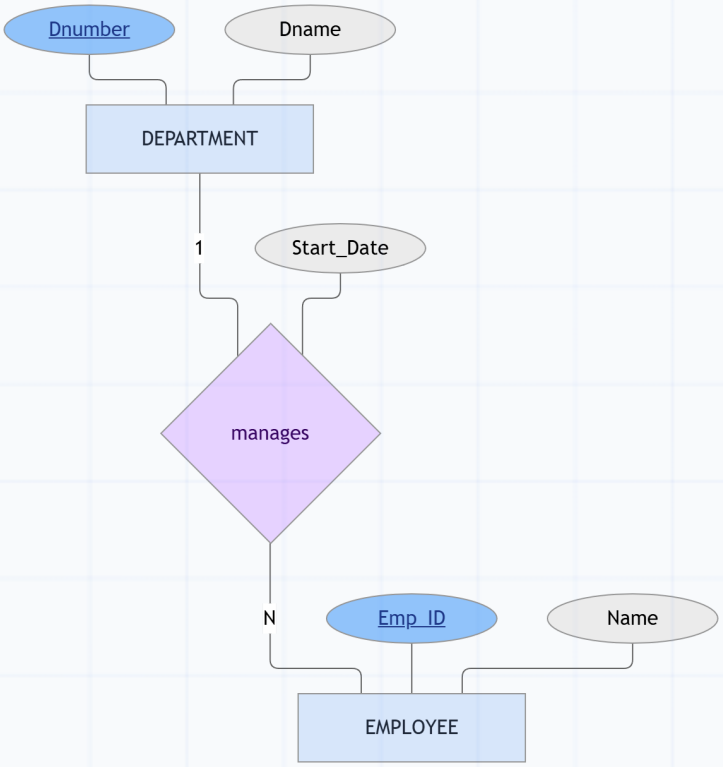


Diagram 49: 1:N Relationship (Partial Participation on N-side) + Rel Attribute

✅ Show Solution

$$\text{DEPARTMENT}(\underline{\text{Dnumber}} : \text{Integer}, \text{Dname} : \text{String})$$
$$\text{IC1: Dname} \neq \text{NULL}$$

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Name} : \text{String},$$
$$\text{Dno} : \text{Integer}, \text{Start_Date} : \text{Date})$$
$$\text{IC2: Dno} \subseteq \text{DEPARTMENT.Dnumber}$$

$$S = \{\text{DEPARTMENT}, \text{EMPLOYEE}\} \cup \{\text{IC1}, \text{IC2}\}$$

> **Note**
>
> `Dnumber` is added to the N-side (`EMPLOYEE`) as `Dno` . The relationship attribute `Start_Date` is placed alongside it. Because participation is Partial, `Dno` is allowed to be `NULL` (no $\neq$ `NULL` constraint).

5.4.2 Case 2: 1:N Relationship (Total Participation on N-side) + Rel Attribute

Rules to Apply:

- **Structural Rule:** Add the 1-side PK as a Foreign Key in the N-side table.
- **Attribute Rule:** Place the relationship attribute in the N-side table.

- **Constraint Rule:** Because the N-side is Total, the FK **must exist**. Add a `NOT NULL` constraint to the FK.

EXAMPLE 15:

A `CUSTOMER` (1) places `ORDER` s (N). The relationship `PLACES` has an attribute `Order_Status`. Every single order **must exist** for a valid customer. A customer **may or may not** have placed orders.

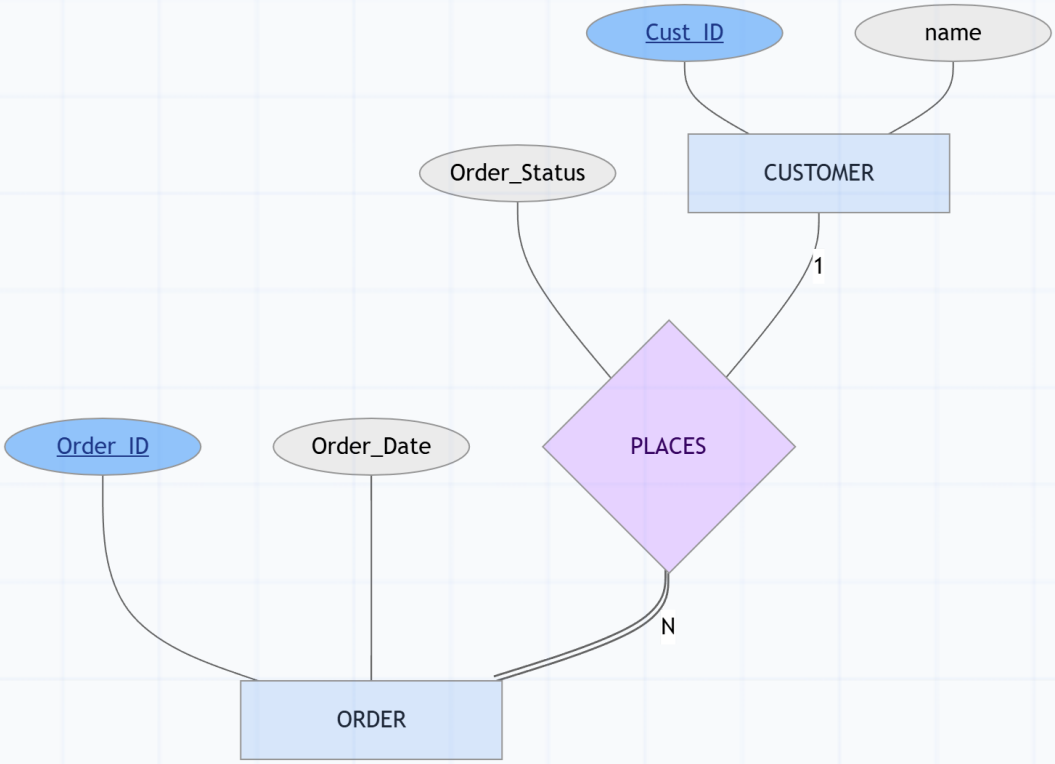


Diagram 50: A CUSTOMER (1) places ORDERs (N). The relationship PLACES has an attribute Order_Status. Every single order must exist for a valid customer. A customer may or may not have placed orders.

✅ Show Solution

$$\text{CUSTOMER}(\underline{\text{Cust_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{ORDER}(\underline{\text{Order_ID}} : \text{Integer}, \text{Order_Date} : \text{Date},$$
$$\text{Cust_ID} : \text{Integer}, \text{Order_Status} : \text{String})$$
$$\text{IC2: Cust_ID} \neq \text{NULL}$$
$$\text{IC3: Cust_ID} \subseteq \text{CUSTOMER.Cust_ID}$$

$$S = \{\text{CUSTOMER}, \text{ORDER}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}\}$$

> **Note**
>
> Because an order **must exist** for a customer (Total Participation), we explicitly enforce **IC3:** `Cust_ID` $\neq$ `NULL`. The relationship attribute `Order_Status` is placed in the N-side table.

5.4.3 Case 3: 1:1 Relationship (Total on One Side, Partial on Other) + Rel Attribute

Rules to Apply:

- **The Golden Rule:** Always place the Foreign Key on the side with **Total Participation** to avoid unnecessary `NULL` s.
- **Attribute Rule:** Place the relationship attribute in the table that receives the Foreign Key.
- **Constraint Rule:** The FK on the Total side **must exist** (`NOT NULL`).

EXAMPLE 16:

A `PERSON` (1) owns a `PASSPORT` (1). The relationship `OWNS` has an attribute `Issue_Date`. A passport **must exist** for a valid person (Total). A person **may or may not** own a passport (Partial).

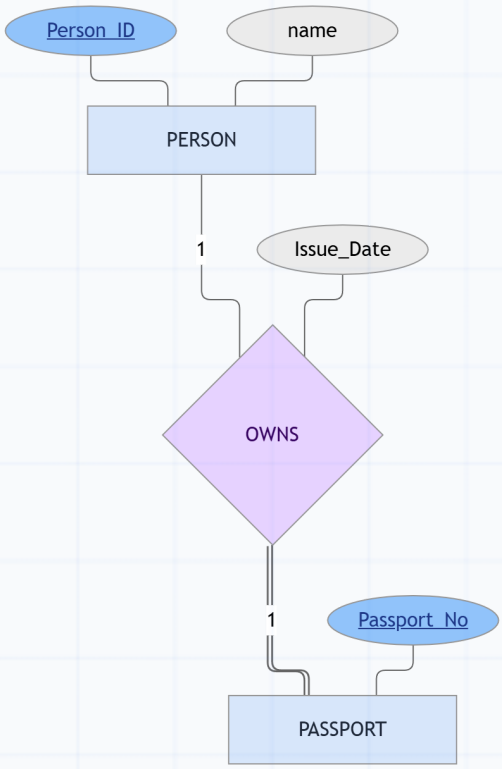


Diagram 51: A PERSON (1) owns a PASSPORT (1). The relationship OWNS has an attribute Issue_Date. A passport must exist for a valid person (Total). A person may or may not own a passport (Partial).

Show Solution

$$\text{PERSON}(\underline{\text{Person_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{PASSPORT}(\underline{\text{Passport_No}} : \text{String},$$
$$\text{Issue_Date} : \text{Date}, \text{Person_ID} : \text{Integer})$$
$$\text{IC2: Person_ID} \neq \text{NULL}$$
$$\text{IC3: Person_ID} \subseteq \text{PERSON.Person_ID}$$

$$S = \{\text{PERSON}, \text{PASSPORT}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}\}$$

> **Note**
>
> Following the Golden Rule, `Person_ID` is placed in `PASSPORT` (the Total side). The relationship attribute `Issue_Date` is also placed there. Because a passport **must exist** with an owner, **IC2** enforces `Person_ID ≠ NULL`.

5.4.4 Case 4: 1:1 Relationship (Partial on BOTH Sides) + Rel Attribute

Rules to Apply:

- **Structural Rule:** Place the FK on either side (structurally valid either way).
- **Attribute Rule:** Place the relationship attribute in the table that receives the FK.
- **Constraint Rule:** Because both sides are Partial, the FK **may or may not** have a value (it must allow `NULL`).

EXAMPLE 17:

An `EMPLOYEE` (1) is assigned a `PARKING_SPOT` (1). The relationship `ASSIGNED` has an attribute `Zone`. An employee **may or may not** have a spot. A spot **may or may not** be assigned to an employee.

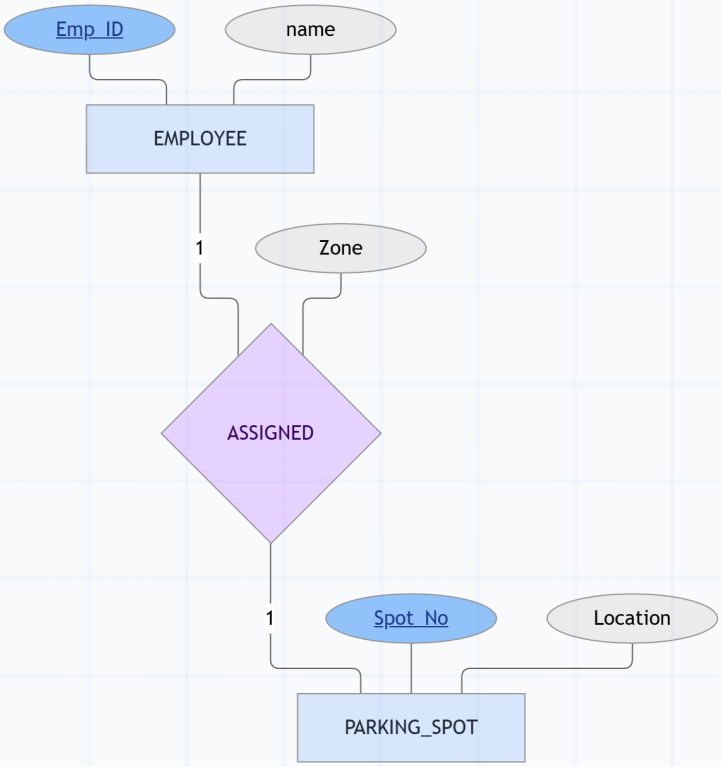


Diagram 52: An EMPLOYEE (1) is assigned a PARKING_SPOT (1). The relationship ASSIGNED has an attribute Zone. An employee may or may not have a spot. A spot may or may not be assigned to an employee.

✅ Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{PARKING_SPOT}(\underline{\text{Spot_No}} : \text{String}, \text{Location} : \text{String},$$
$$\text{Zone} : \text{String}, \text{Emp_ID} : \text{Integer})$$
$$\text{IC2: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$$

$$S = \{\text{EMPLOYEE}, \text{PARKING_SPOT}\} \cup \{\text{IC1}, \text{IC2}\}$$

> ⓘ **Note**
>
> We placed the FK and the relationship attribute `Zone` in `PARKING_SPOT`. Because both sides *may or may not exist* in the relationship, `Emp_ID` is allowed to be `NULL` (no `≠ NULL` constraint).

5.4.5 Case 5: M:N Relationship + Relationship Attribute

Rules to Apply:

- **Structural Rule:** Create a brand new **Junction (Associative) Table**.

- **Key Rule:** The PK of the junction table is the composite of both entities' PKs.
- **Attribute Rule:** The relationship attribute **must** be placed inside the new junction table.

EXAMPLE 18:

A `STUDENT` (M) enrolls in a `COURSE` (N). The relationship `ENROLLS_IN` has an attribute `Grade`. A student **may or may not** be enrolled. A course **may or may not** have students.

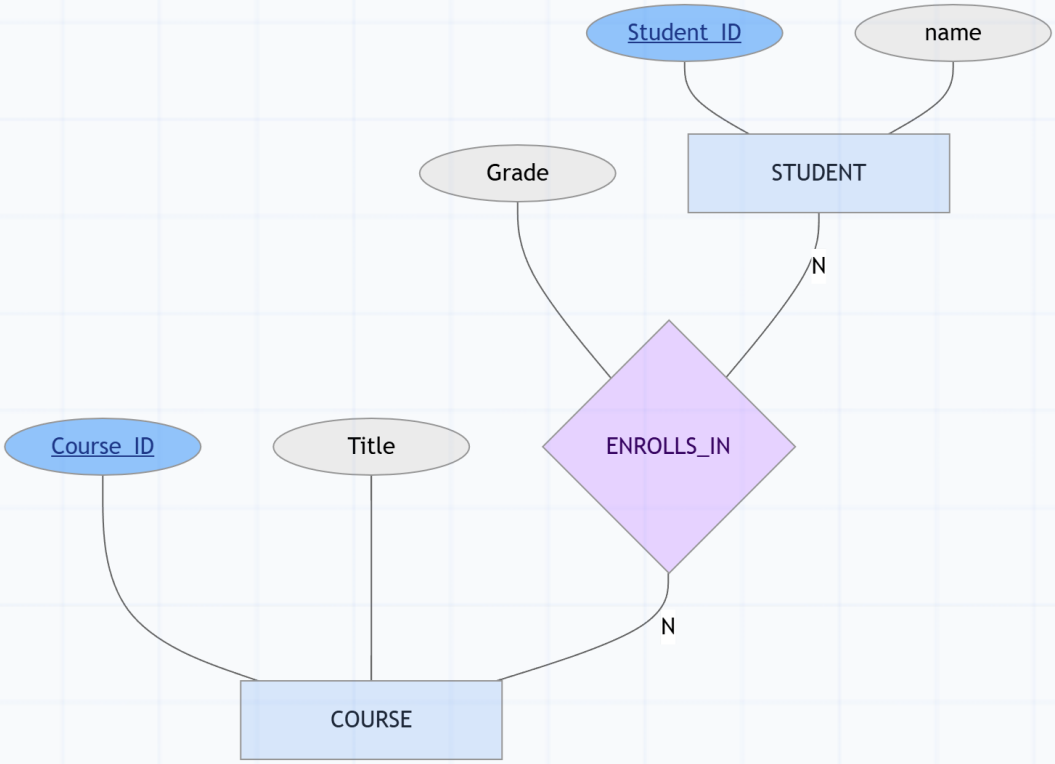


Diagram 53: A STUDENT (M) enrolls in a COURSE (N). The relationship ENROLLS_IN has an attribute Grade. A student may or may not be enrolled. A course may or may not have students.

Show Solution

$$\text{STUDENT}(\underline{\text{Student_ID}} : \text{Integer}, \text{Name} : \text{String})$$
$$\text{IC1: Name} \neq \text{NULL}$$

$$\text{COURSE}(\underline{\text{Course_ID}} : \text{String}, \text{Title} : \text{String})$$
$$\text{IC2: Title} \neq \text{NULL}$$

$$\text{ENROLLMENT}(\underline{\text{Student_ID}} : \text{Integer},$$
$$\underline{\text{Course_ID}} : \text{String}, \text{Grade} : \text{String})$$
$$\text{IC3: } \mathit{Grade} \in \{'A', 'B', 'C', 'D', 'F'\}$$
$$\text{IC4: Student_ID} \subseteq \text{STUDENT.Student_ID}$$
$$\text{IC5: Course_ID} \subseteq \text{COURSE.Course_ID}$$

$$S = \{\text{STUDENT}, \text{COURSE}, \text{ENROLLMENT}\} \cup \{\text{IC1}, \text{IC2}, \text{IC3}, \text{IC4}, \text{IC5}\}$$

> **Note**
>
> The M:N relationship is resolved via the `ENROLLMENT` junction table. The relationship attribute `Grade` lives exclusively in this new table. The composite PK is (`Student_ID` , `Course_ID`).

5.4.6 Case 6: Unary (Recursive) 1:N Relationship + Rel Attribute

Rules to Apply:

- **Structural Rule:** Add a Foreign Key to the **same table**, referencing its own Primary Key.
- **Attribute Rule:** Place the relationship attribute in the same table alongside the recursive Foreign Key.

EXAMPLE 19:

The `EMPLOYEE` entity participates in a unary 1:N relationship `SUPERVISION` . An employee (as a manager) supervises other employees. The relationship has an attribute `Start_Date` . An employee **may or may not** have a supervisor (e.g., the CEO has none).

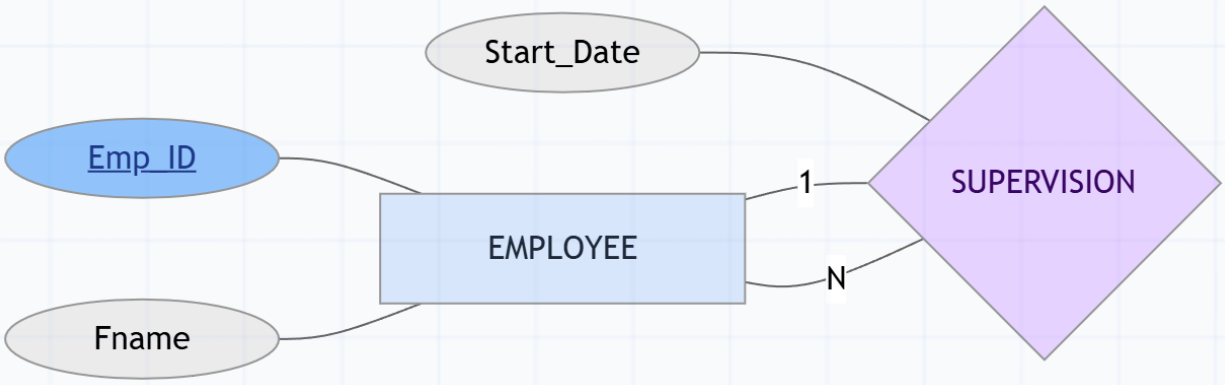


Diagram 54: The EMPLOYEE entity participates in a unary 1:N relationship SUPERVISION. An employee (as a manager) supervises other employees. The relationship has an attribute Start_Date. An employee may or may not have a supervisor (e.g., the CEO has none).

> **Show Solution**

$$\begin{aligned}&\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Fname} : \text{String},\\ &\quad \text{Super_Emp_ID} : \text{Integer}, \text{Start_Date} : \text{Date})\\ &\text{IC1: Fname} \neq \text{NULL}\\ &\text{IC2: Super_Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}\\ &S = \{\text{EMPLOYEE}\} \cup \{\text{IC1}, \text{IC2}\}\end{aligned}$$

> **Note**
>
> `Super_Emp_ID` is added to the `EMPLOYEE` table to point to `Emp_ID`. The relationship attribute `Supervision_Start_Date` is placed right next to it. Because the CEO **may or may not** have a supervisor, `Super_Emp_ID` is allowed to be `NULL`.

5.4.7 Case 7: Weak Entity (Identifying Relationship) + Partial Key

Rules to Apply:

- **Structural Rule:** Map the weak entity to a new table.
- **Key Rule:** The Primary Key is **Composite**, formed by the owner entity's PK + the weak entity's partial key (discriminator). Both are underlined.
- **Constraint Rule:** The owner's PK acts as a Foreign Key and **must exist** (`NOT NULL`) because a weak entity **must exist** with its owner (Total participation in the identifying relationship).

EXAMPLE 20:

`DEPENDENT` is a weak entity of `EMPLOYEE`. It has a partial key `Dep_Name`. A dependent **must exist** for an employee (Total participation). An employee **may or may not** have dependents (Partial participation).

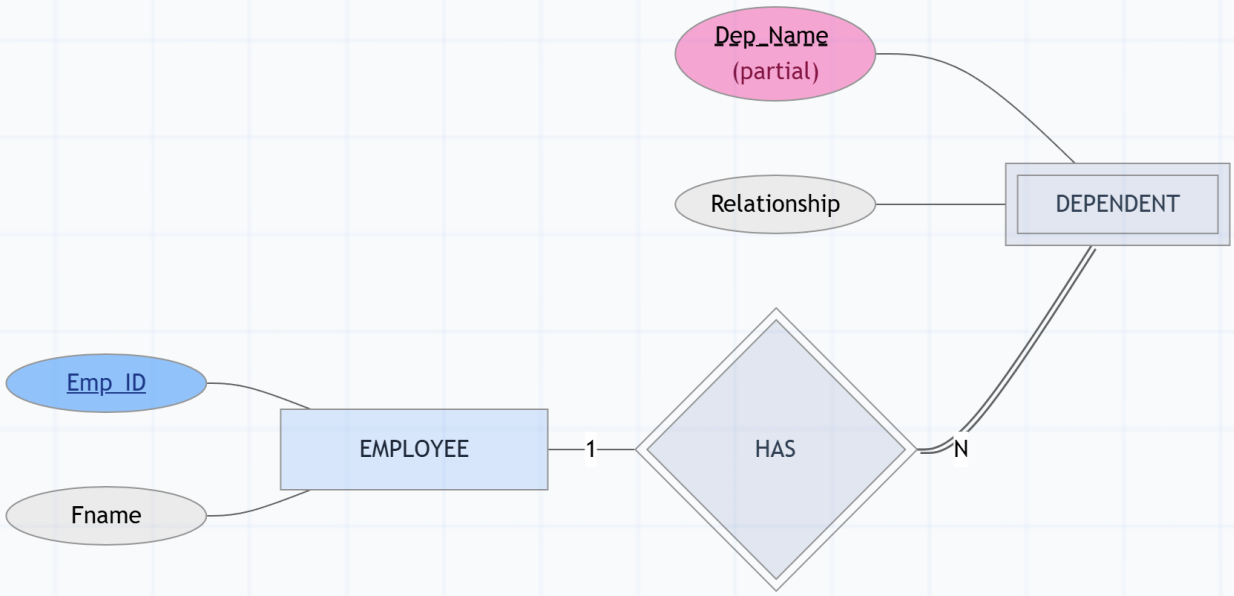


Diagram 55: DEPENDENT is a weak entity of EMPLOYEE. It has a partial key Dep_Name. A dependent must exist for an employee (Total participation). An employee may or may not have dependents (Partial participation).

✅ Show Solution

$$\text{EMPLOYEE}(\underline{\text{Emp_ID}} : \text{Integer}, \text{Fname} : \text{String})$$
$$\text{IC1: Fname} \neq \text{NULL}$$

$$\text{DEPENDENT}(\underline{\text{Emp_ID}} : \text{Integer},$$
$$\underline{\text{Dep_Name}} : \text{String},$$
$$\text{Relationship} : \text{String})$$
$$\text{IC2: } \mathit{Relationship} \in \{'\mathit{Spouse}', '\mathit{Son}', '\mathit{Daughter}'\}$$
$$\text{IC3: Emp_ID} \subseteq \text{EMPLOYEE.Emp_ID}$$

$$S = \{\text{EMPLOYEE, DEPENDENT}\} \cup \{\text{IC1, IC2, IC3}\}$$

> **Note**
>
> The PK of `DEPENDENT` is composite (`Emp_ID` , `Dep_Name`). Both are underlined. `Emp_ID` is underlined because it is part of the PK, and it simultaneously acts as the Foreign Key. Since `Emp_ID` is already **underlined** as part of the Composite Primary Key of `DEPENDENT` , the Entity Integrity Constraint **automatically** guarantees that `Emp_ID` $\neq$ `NULL` .